

# Gramática del lenguaje BattleScript (BNF)

Organización de Lenguajes y Compiladores 1 – Proyecto 1

Zabdi Merari Adina Donis Rosales - 202308487

## Contenido

|                                             |   |
|---------------------------------------------|---|
| 1. Notación                                 | 1 |
| 2. Símbolo inicial                          | 1 |
| 3. Sección de estrategias                   | 2 |
| 4. Sección de partidas                      | 2 |
| 5. Punto de entrada                         | 3 |
| 6. Expresiones                              | 3 |
| 7. Funciones del sistema                    | 4 |
| 8. Clases léxicas (terminales no literales) | 4 |
| 9. Ejemplo derivable con esta gramática     | 4 |
| 10. Notas de implementación                 | 5 |

# 1. Notación

Se utiliza la notación **BNF (Backus-Naur Form)**:

- `<no_terminal>` – símbolos no terminales.
- `"texto"` – símbolos terminales literales (palabras reservadas y operadores).
- `::=` – regla de derivación (“se define como”).
- `|` – alternativa (“o”).
- $\epsilon$  – cadena vacía.
- Los terminales `ID`, `ENTERO`, `DECIMAL`, `MAGE_ACTION` y `WARRIOR_ACTION` representan clases léxicas (identificador, entero, decimal, acción de mago y acción de guerrero, respectivamente) y se describen en la sección 6.

## 2. Símbolo inicial

```
<programa> ::= <lista_estrategias> <lista_partidas> <main>
```

## 3. Sección de estrategias

```
<lista_estrategias> ::= <estrategia>
                        | <lista_estrategias> <estrategia>
                        |  $\epsilon$ 
```

```
<estrategia> ::= "mage" ID "{" "initial" ":" <accion_mago> <reglas_mago> "}"
                | "warrior" ID "{" "initial" ":" <accion_guerrero> <reglas_guerrero> "}"
```

```
<accion_mago>      ::= MAGE_ACTION
<accion_guerrero> ::= WARRIOR_ACTION
```

```
<reglas_mago> ::= "rules" ":" "[" <lista_reglas_mago> "else" <accion_mago> "]"
                | "rules" ":" "[" <lista_reglas_mago> "," "else" <accion_mago> "]"
```

```
<lista_reglas_mago> ::= <regla_mago>
                        | <lista_reglas_mago> "," <regla_mago>
                        |  $\epsilon$ 
```

```
<regla_mago> ::= "if" <expresion> "then" <accion_mago>
```

```
<reglas_guerrero> ::= "rules" ":" "[" <lista_reglas_guerrero> "else" <accion_guerrero> "]"
                    | "rules" ":" "[" <lista_reglas_guerrero> "," "else" <accion_guerrero> "]"
```

```
<lista_reglas_guerrero> ::= <regla_guerrero>
                           | <lista_reglas_guerrero> "," <regla_guerrero>
                           |  $\epsilon$ 
```

```
<regla_guerrero> ::= "if" <expresion> "then" <accion_guerrero>
```

## 4. Sección de partidas

```
<lista_partidas> ::= <partida>
                  | <lista_partidas> <partida>
                  | ε

<partida> ::= "match" ID "{"
             "players" ":" "[" ID "," ID "]"
             "rounds" ":" ENTERO
             <bloque_scoring>
             <bonuses_opcional>
             "}"

<bloque_scoring> ::= "scoring" ":" "{"
                    "damage_point" ":" ENTERO ","
                    "healing_point" ":" ENTERO ","
                    "successful_defense" ":" ENTERO ","
                    "victory_bonus" ":" ENTERO ","
                    "failed_action_penalty" ":" ENTERO
                    "}"

<bonuses_opcional> ::= <bloque_bonuses>
                     | ε

<bloque_bonuses> ::= "bonuses" ":" "{"
                    "mage_combo" ":" "[" <lista_acciones> "]" ","
                    "mage_combo_points" ":" ENTERO ","
                    "warrior_combo" ":" "[" <lista_acciones> "]" ","
                    "warrior_combo_points" ":" ENTERO ","
                    "low_health_victory" ":" ENTERO
                    "}"

<lista_acciones> ::= <accion_mago>
                   | <lista_acciones> "," <accion_mago>
                   | <accion_guerrero>
                   | <lista_acciones> "," <accion_guerrero>
```

## 5. Punto de entrada

```
<main> ::= "main" "{" <lista_run> "}"

<lista_run> ::= <instruccion_run>
              | <lista_run> <instruccion_run>
              | ε

<instruccion_run> ::= "run" "[" <lista_ids> "]" "with" "{" "seed" ":" ENTERO "}"

<lista_ids> ::= ID
              | <lista_ids> "," ID
```

## 6. Expresiones

La precedencia (de menor a mayor) es: || < && < comparaciones (==, !=, <, >, <=, >=, sin asocia-

tividad) < ! < paréntesis / funciones / literales.

```
<expresion> ::= <expresion_or>

<expresion_or> ::= <expresion_or> "||" <expresion_and>
                | <expresion_and>

<expresion_and> ::= <expresion_and> "&&" <expresion_comparacion>
                  | <expresion_comparacion>

<expresion_comparacion> ::= <expresion_unaria> "==" <expresion_unaria>
                           | <expresion_unaria> "!=" <expresion_unaria>
                           | <expresion_unaria> "<" <expresion_unaria>
                           | <expresion_unaria> ">" <expresion_unaria>
                           | <expresion_unaria> "<=" <expresion_unaria>
                           | <expresion_unaria> ">=" <expresion_unaria>
                           | <expresion_unaria>

<expresion_unaria> ::= "!" <expresion_unaria>
                    | <expresion_primaria>

<expresion_primaria> ::= ENTERO
                     | DECIMAL
                     | MAGE_ACTION
                     | WARRIOR_ACTION
                     | "[" <lista_acciones> "]"
                     | "self_health"
                     | "opponent_health"
                     | "self_resource"
                     | "opponent_resource"
                     | "self_score"
                     | "opponent_score"
                     | "round_number"
                     | "total_rounds"
                     | "random"
                     | ID
                     | <funcion>
                     | "(" <expresion> ")"
```

## 7. Funciones del sistema

```
<funcion> ::= "get_moves_count" "(" <historial> "," <expresion> ")"
            | "get_moves_count" "(" <funcion> "," <expresion> ")"
            | "get_move" "(" <historial> "," ENTERO ")"
            | "last_move" "(" <historial> ")"
            | "get_last_n_moves" "(" <historial> "," ENTERO ")"

<historial> ::= "self_history"
              | "opponent_history"
```

## 8. Clases léxicas (terminales no literales)

| Terminal       | Descripción                                                            | Ejemplo            |
|----------------|------------------------------------------------------------------------|--------------------|
| ID             | Identificador: una letra seguida de letras, dígitos o guion bajo.      | Merlin, DueloFinal |
| ENTERO         | Secuencia de uno o más dígitos.                                        | 42, 100            |
| DECIMAL        | Uno o más dígitos, punto, uno o más dígitos.                           | 0.25               |
| MAGE_ACTION    | Una de: ARCANES_BOLT, FIREBALL, MAGIC_BARRIER, HEALING_RUNE, MEDITATE. | FIREBALL           |
| WARRIOR_ACTION | Una de: SLASH, HEAVY_STRIKE, SHIELD_BLOCK, WAR_CRY, REST.              | SLASH              |

## 9. Ejemplo derivable con esta gramática

```
mage Merlin {
  initial: ARCANES_BOLT
  rules: [
    if self_health <= 30 && self_resource >= 30 then HEALING_RUNE,
    if opponent_health <= 40 then FIREBALL,
    else ARCANES_BOLT
  ]
}
warrior Ragnar {
  initial: SLASH
  rules: [
    if self_resource <= 20 then REST,
    else SLASH
  ]
}
match DueloInicial {
  players: [Merlin, Ragnar]
  rounds: 5
  scoring: {
    damage_point: 1,
    healing_point: 1,
    successful_defense: 20,
    victory_bonus: 100,
    failed_action_penalty: 10
  }
  bonuses: {
    mage_combo: [ARCANES_BOLT, ARCANES_BOLT, FIREBALL],
    mage_combo_points: 35,
    warrior_combo: [SLASH, SLASH, HEAVY_STRIKE],
    warrior_combo_points: 40,
    low_health_victory: 25
  }
}
main {
  run [DueloInicial] with {
    seed: 42
  }
}
```

## 10. Notas de implementación

- La restricción de que las acciones usadas en `initial`, `then` y `else` correspondan a la clase de la estrategia (`mage` → `MAGE_ACTION`, `warrior` → `WARRIOR_ACTION`) **se aplica a nivel léxico/sintáctico** mediante los terminales distintos `MAGE_ACTION` / `WARRIOR_ACTION`, y se refuerza a nivel semántico comparando `Action.getOwner()` contra el `ClassType` de la estrategia.
- El bloque `else` de `<reglas_mago>` / `<reglas_guerrero>` debe aparecer **exactamente una vez y siempre al final**; esto queda garantizado por la propia forma de la regla (no existe ninguna producción que permita otro `else` o una regla `if` después del `else`).
- Esta gramática es una versión limpia y documentada, escrita a partir del archivo fuente `src/cup/Parser.cup`, y no es una copia literal de dicho archivo.